In [1]:
```
import numpy as np
import pandas as pd
#from tensorflow.keras.datasets import fashion_mnist
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score
```

In [2]:
```
import pandas as pd

df = pd.read_csv('/home/backup/ml_datasets/fashion_mnist.csv')
#print(df.head(5))
```

In [3]:
```
X = df.drop('class', axis=1)
y = df[['class']]
```

In [4]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=4

print("X_train shape: ", X_train.shape)
print("y_train shape: ", y_train.shape)
print("X_test shape: ", X_test.shape)
print("y_test shape: ", y_test.shape)
```

```
X_train shape:  (56000, 784)
y_train shape:  (56000, 1)
X_test shape:  (14000, 784)
y_test shape:  (14000, 1)
```

In [5]:
```
from sklearn.neighbors import KNeighborsClassifier
#experiment with diff k

k_values = [1,3,5]

for k in k_values:
    print(f"\n")
    print(f"\n ===== K = {k} =====")

    knn = KNeighborsClassifier(n_neighbors = k, n_jobs = -1)
    knn.fit(X_train[:10000], y_train[:10000]) #subset for speed

    y_pred = knn.predict(X_test[:2000])
    acc = accuracy_score(y_test[:2000],y_pred)

    print("\n Accuracy:",acc)
    print(classification_report(y_test[:2000], y_pred))
```

```
 ===== K = 1 =====
/usr/local/lib64/python3.6/site-packages/sklearn/neighbors/_classification.py:179: Da
taConversionWarning: A column-vector y was passed when a 1d array was expected. Pleas
e change the shape of y to (n_samples,), for example using ravel().
  return self._fit(X, y)
 Accuracy: 0.8045
              precision    recall  f1-score   support

           0       0.75      0.80      0.77       211
```

```
           1       0.99      0.95      0.97       172
           2       0.69      0.67      0.68       190
           3       0.88      0.84      0.86       209
           4       0.65      0.68      0.67       193
           5       0.98      0.78      0.87       205
           6       0.52      0.53      0.53       212
           7       0.84      0.93      0.88       191
           8       0.97      0.93      0.95       200
           9       0.86      0.96      0.91       217

    accuracy                           0.80      2000
   macro avg       0.81      0.81      0.81      2000
weighted avg       0.81      0.80      0.81      2000
```

```
 ===== K = 3 =====
/usr/local/lib64/python3.6/site-packages/sklearn/neighbors/_classification.py:179: Da
taConversionWarning: A column-vector y was passed when a 1d array was expected. Pleas
e change the shape of y to (n_samples,), for example using ravel().
  return self._fit(X, y)
 Accuracy: 0.8195
              precision    recall  f1-score   support

           0       0.72      0.88      0.79       211
           1       0.97      0.95      0.96       172
           2       0.63      0.69      0.66       190
           3       0.91      0.85      0.88       209
           4       0.71      0.69      0.70       193
           5       0.98      0.80      0.88       205
           6       0.59      0.50      0.54       212
           7       0.87      0.95      0.91       191
           8       0.98      0.93      0.95       200
           9       0.89      0.98      0.93       217

    accuracy                           0.82      2000
   macro avg       0.82      0.82      0.82      2000
weighted avg       0.82      0.82      0.82      2000
```

```
 ===== K = 5 =====
/usr/local/lib64/python3.6/site-packages/sklearn/neighbors/_classification.py:179: Da
taConversionWarning: A column-vector y was passed when a 1d array was expected. Pleas
e change the shape of y to (n_samples,), for example using ravel().
  return self._fit(X, y)
 Accuracy: 0.8185
              precision    recall  f1-score   support

           0       0.74      0.87      0.80       211
           1       0.96      0.95      0.96       172
           2       0.67      0.72      0.69       190
           3       0.88      0.84      0.86       209
           4       0.71      0.71      0.71       193
           5       0.98      0.78      0.86       205
           6       0.55      0.49      0.52       212
           7       0.86      0.96      0.91       191
           8       0.98      0.92      0.95       200
           9       0.89      0.98      0.93       217

    accuracy                           0.82      2000
   macro avg       0.82      0.82      0.82      2000
weighted avg       0.82      0.82      0.82      2000
```

In [ ]: